Implementation Task 3 (IT3)

Due: Sunday, 24 May 2026, by 9:00 PM (Singapore time)
Weight: 4% of total assessment

Overview

This task involves the use of **MongoDB** for:

- Reverse engineering BSON documents
- Writing queries for data retrieval and manipulation
- Demonstrating understanding of hierarchical document structure

All implementations and outputs must be submitted **electronically via Moodle**.

Note: Submissions via email will be **deleted** and receive a **mark of zero**.

Submission Requirements

You must submit the following three files in a single zipped folder:

File Name	Content
IT3Task1.js	MongoDB queries and data manipulation (Task 1)
IT3Task1Solution.txt	Terminal output from running IT3Task1.js

Name your zipped file as:

<YourUOWStudentNumber>-IT3.zip

Upload the ZIP to **Moodle** under the *Implementation Task 3* submission section.

Preliminary Task (No marks)

Download and run the script customerOrder.js:

mongosh "mongodb://localhost:27017/myDB" --file
"D:/mongoDBScripts/customerOrder.js"

Verify data was loaded using:
db.customerOrder.find().pretty()

No submission or report required for this task.

Task 1: MongoDB Data Manipulation (4.0 marks)

Instructions:

Using db.customerOrder collection, complete the following tasks in a script file IT3Task1.js. Then run it and **save the full output** (commands + results) to IT3Task1Solution.txt.

Mark for each subtask are indicated for next to the questions.

Tasks to Implement:

Use either a method $find()$ or a method $aggregate()$ available in MongoDB to write the implementations of the following queries.

(i) `For each customer, display firstName, lastName, street, postalCode, and country. Do not show the object identifier (_id).`

(0.6 mark)

```
csci235> // For each customer, display firstName, lastName, street,
postalCode, and country
| db.customerOrder.find(
|   {},  // No filter - get all customers
|   {
|     firstName: 1,
|     lastName: 1,
|     "address.street": 1,
|     "address.postalCode": 1,
|     "address.country": 1,
|     _id: 0  // Exclude the object identifier
|   }
| );
[
  {
    firstName: 'Andrew',
    lastName: 'Smith',
```

```
   address: { street: 'Clementi Road', postalCode: '599491', country: 'Singapore' }
  }
]
csci235>
```

(ii) Find the first name and last name of the customer who made order "ord001".

(0.6 mark)

```
csci235> // Find the first name and last name of the customer who made order "ord001"
| db.customerOrder.find(
|   { "orders.orderNumber": "ord001" },  // Query for order number
|   {
|     firstName: 1,
|     lastName: 1,
|     _id: 0  // Exclude the object identifier
|   }
| );
[ { firstName: 'Andrew', lastName: 'Smith' } ]
csci235>
```

(iii) Display one result per order with firstName, lastName, email, orderNumber, orderDate, staffNumber.

(0.8 mark)

```
csci235> // Display one result per order with firstName, lastName, email, orderNumber, orderDate, staffNumber
| db.customerOrder.aggregate([
|   {
|     $unwind: "$orders"  // Deconstruct the orders array to create one document per order
```

```
|   },
|   {
|     $project: {
|       firstName: 1,
|       lastName: 1,
|       email: 1,
|       orderNumber: "$orders.orderNumber",
|       orderDate: "$orders.orderDate",
|       staffNumber: "$orders.staffNumber",
|       _id: 0  // Exclude the object identifier
|     }
|   }
| ]);
[
  {
    firstName: 'Andrew',
    lastName: 'Smith',
    email: 'aSmith@gmail.com',
    orderNumber: 'ord001',
    orderDate: '10-MAY-2019',
    staffNumber: 'stf123'
  },
  {
    firstName: 'Andrew',
    lastName: 'Smith',
    email: 'aSmith@gmail.com',
    orderNumber: 'ord005',
    orderDate: '15-MAY-2019',
    staffNumber: 'stf890'
  }
]
csci235>
```

Use the method $update()$ to write the implementations of the following data manipulation operations. Implementation of each data manipulation operation is worth 1 mark.

(iv) Add a new field called membershipLevel: "Silver" to Andrew Smith's document.

(1.0 mark)

```
csci235> // Add a new field called membershipLevel: "Silver" to Andrew Smith's document
| db.customerOrder.updateOne(
|   { firstName: "Andrew", lastName: "Smith" },  // Filter to find Andrew Smith
|   { $set: { membershipLevel: "Silver" } }      // Add the new field
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
csci235>
```

(v) Update Andrew Smith's email address to Andrew.smith@gmail.com.

(1.0 mark)

```
csci235> // Update Andrew Smith's email address to Andrew.smith@gmail.com
| db.customerOrder.updateOne(
|   { firstName: "Andrew", lastName: "Smith" },  // Filter to find Andrew Smith
|   { $set: { email: "Andrew.smith@gmail.com" } } // Update the email field
```

```
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
csci235>
```

Deliverables:

- IT3Task1.js: contains the above MongoDB commands
- IT3Task1Solution.txt: copy-pasted output from terminal showing:
  - all MongoDB queries
  - their output using pretty()
  - no syntax errors
  -

Important: Do not omit the MongoDB commands in the output file. A report without query listings will receive **no marks**.

Sample solution includes but is not limited to the following:

```
csci235> db.customerOrder.find()
[
  {
    _id: ObjectId('6a17b7db13bea60a1b3682d1'),
    firstName: 'Andrew',
    lastName: 'Smith',
    email: 'aSmith@gmail.com',
    address: {
      street: 'Clementi Road',
      houseNumber: '123',
```

```
    postalCode: '599491',
    country: 'Singapore'
  },
  orders: [
    {
      orderNumber: 'ord001',
      orderDate: '10-MAY-2019',
      staffNumber: 'stf123',
      lineItem: [
        {
          lineNum: 1,
          productCode: 'p001',
          productDesc: 'Note-book',
          orderQty: 2,
          price: { currency: 'SGD', value: 4800 }
        },
        {
          lineNum: 2,
          productCode: 's005',
          productDesc: '4TB Hard-disk',
          orderQty: 1,
          price: { currency: 'USD', value: 350 }
        },
        {
          lineNum: 3,
          productCode: 'a001',
          productDesc: 'ear-piece',
          orderQty: 1,
          price: { currency: 'SGD', value: 10 }
        }
      ]
    },
    {
      orderNumber: 'ord005',
      orderDate: '15-MAY-2019',
      staffNumber: 'stf890',
      lineItem: [
        {
          lineNum: 1,
          productCode: 'p002',
          productDesc: 'Personal Computer',
          orderQty: 1,
          price: { currency: 'SGD', value: 1400 }
```

```
     },
     {
      lineNum: 2,
      productCode: 'a005',
      productDesc: 'Laser Printer',
      orderQty: 1,
      price: { currency: 'USD', value: 250 }
     }
    ]
   }
  ]
 }
]
csci235>
```

Submissions

This implementation task is due by 9:00 pm (2100 hours) Sunday, 24 May 2026, Singapore time.

Submit the files **IT3Task1.js,** and **IT3Task1Solution.txt** through Moodle in the following way:

1) Zip all the files (**IT3Task2.js** and **IT3Task2Solution.txt** into one zipped folder. Name your zipped file as YourName-IT3)
2) Access Moodle at **http://moodle.uowplatform.edu.au/**
3) To login use a Login link located in the right upper corner the Web page or in the middle of the bottom of the Web page
4) When successfully logged in, select a site CSCI235 (SP226) Database Systems
5) Scroll down to a section Submissions of Implementation Tasks
6) Click at Submit your Implementation Task 3 here link.
7) Click at a button Add Submission
8) Move the zipped file created in Step 1 above into an area provided in Moodle. You can drag and drop files here to add them. You can also use a link *Add…*
9) Click at a button Save changes,
10) Click at check box to confirm authorship of a submission,
11) When you are satisfied, remember to click at a button Submit assignment.

A policy regarding late submissions is included in the subject outline. Only one submission per student is accepted.

Implementation Task 3 is an individual assignment, and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

End of specification